

Leftist Tree

左偏树

什么是左偏树？

左偏树与 [配对堆](#) 一样，是一种 **可并堆**，具有堆的性质，并且可以快速合并。

左偏树的定义和性质

对于一棵二叉树，我们定义 **外节点** 为子节点数小于两个的节点，定义一个节点的 df 为其到子树中最近的外节点所经过的边的数量。空节点的 df 为 0。

▼ 注意

有些资料中对 df 的定义是本文中的 df 减 1，这样定义是因为代码编写时可以省略一些判空流程，但需要注意应预先置空节点的 df 为 0。本文中所有代码对 df 的定义 **均为后者**，请注意与行文间 df 定义的差别。

左偏树是一棵二叉树，它不仅具有堆的性质，并且是「左偏」的：每个节点左儿子的 df 都大于等于右儿子的 df 。

因此，左偏树每个节点的 df 都等于其右儿子的 df 加一。

需要注意的是， df 不是深度，左偏树的深度没有保证，一条向左的链也符合左偏树的定义。

核心操作：合并（merge）

合并两个堆时，由于要满足堆性质，先取值较小（为了方便，本文讨论小根堆）的那个根作为合并后堆的根节点，然后将这个根的左儿子作为合并后堆的左儿子，递归地合并其右儿子与另一个堆，作为合并后的堆的右儿子。为了满足左偏性质，合并后若左儿子的 df 小于右儿子的 df ，就交换两个儿子。

参考代码：

▼ 实现

```
1 int merge(int x, int y) {
2   if (!x || !y) return x | y; // 若一个堆为空则返回另一个堆
3   if (t[x].val > t[y].val) swap(x, y); // 取值较小的作为根
4   t[x].rs = merge(t[x].rs, y); // 递归合并右儿子与另一个堆
5   if (t[t[x].rs].d > t[t[x].ls].d)
6     swap(t[x].ls, t[x].rs); // 若不满足左偏性质则交换左右儿子
7   t[x].d = t[t[x].rs].d + 1; // 更新dist
8   return x;
9 }
```

由于左偏性质，每递归一层，其中一个堆根节点的 df 就会减小，而一棵有 n 个节点的二叉树，根的 df 不超过 $\log n$ ，所以合并两个大小分别为 n 和 m 的堆复杂度是 $O(\log n + \log m)$ 。

▼ 关于 df 性质的证明

一棵根的 df 为 d 的二叉树至少有 2^d 层是满二叉树，那么就至少有 2^d 个节点。注意这个性质是所有二叉树都具有的，并不是左偏树所特有的。

左偏树还有一种无需交换左右儿子的写法：将较大的儿子视作左儿子，较小的儿子视作右儿子：

▼ 实现

```
1 int& rs(int x) { return t[x].ch[t[t[x].ch[1]].d < t[t[x].ch[0]].d]; }
2
3 int merge(int x, int y) {
4     if (!x || !y) return x | y;
5     if (t[x].val < t[y].val) swap(x, y);
6     int& rs_ref = rs(x);
7     rs_ref = merge(rs_ref, y);
8     t[x].d = t[rs(x)].d + 1;
9     return x;
10 }
```

左偏树的其它操作

插入节点

单个节点也可以视为一个堆，合并即可。

删除根

合并根的左右儿子即可。

删除任意节点

做法

先将左右儿子合并，然后自底向上更新、不满足左偏性质时交换左右儿子，当 无需更新时结束递归：

▼ 实现

```

1 int& rs(int x) { return t[x].ch[t[t[x].ch[1]].d < t[t[x].ch[0]].d]; }
2
3 // 有了 pushup, 直接 merge 左右儿子就实现了删除节点并保持左偏性质
4 int merge(int x, int y) {
5     if (!x || !y) return x | y;
6     if (t[x].val < t[y].val) swap(x, y);
7     int& rs_ref = rs(x);
8     rs_ref = merge(rs_ref, y);
9     t[rs_ref].fa = x;
10    t[x].d = t[rs(x)].d + 1;
11    return x;
12 }
13
14 void pushup(int x) {
15     if (!x) return;
16     if (t[x].d != t[rs(x)].d + 1) {
17         t[x].d = t[rs(x)].d + 1;
18         pushup(t[x].fa);
19     }
20 }
21
22 void erase(int x) {
23     int y = merge(t[x].ch[0], t[x].ch[1]);
24     t[y].fa = t[x].fa;
25     if (t[t[x].fa].ch[0] == x)
26         t[t[x].fa].ch[0] = y;
27     else if (t[t[x].fa].ch[1] == x)
28         t[t[x].fa].ch[1] = y;
29     pushup(t[y].fa);
30 }

```

复杂度证明

先考虑 `merge` 的过程，每次都会使 `x` 或 `y` 向下一层，也就是说最极端的情况，就是一直选择左偏树的右节点（最小的节点）向下一层，此时 `d` 减少了。

再考虑 `pushup` 的过程，我们令当前 `pushup` 的这个节点为 `x`，其父亲为 `fa`，一个节点的「初始」为它在 `pushup` 前的 `d`。从被删除节点的父亲开始递归，有两种情况：

1. `x` 是 `fa` 的右儿子，此时 `x` 的初始为 `fa` 的初始加一。
2. `x` 是 `fa` 的左儿子，由于节点的 `d` 最多减一，因此只有 `x` 的左右儿子初始相等时（此时左儿子 `d` 减一会导致左右儿子互换）才会继续递归下去，因此 `fa` 的初始仍然是 `x` 的初始加一。

所以，我们得到，每递归一层 `fa` 的初始就会加一，因此最多递归 `n` 层。

整个堆加上/减去一个值、乘上一个正数

其实可以打标记且不改变相对大小的操作都可以。

在根打上标记，删除根/合并堆（访问儿子）时上传标记即可：

▼ 实现

```

1 int merge(int x, int y) {
2     if (!x || !y) return x | y;
3     if (t[x].val > t[y].val) swap(x, y);
4     pushdown(x);
5     t[x].rs = merge(t[x].rs, y);
6     if (t[t[x].rs].d > t[t[x].ls].d) swap(t[x].ls, t[x].rs);
7     t[x].d = t[t[x].rs].d + 1;
8     return x;
9 }
10
11 int pop(int x) {
12     pushdown(x);
13     return merge(t[x].ls, t[x].rs);
14 }

```

其他可并堆

随机堆

▼ 实现

```

1 int merge(int x, int y) {
2     if (!x || !y) return x | y;
3     if (t[y].val < t[x].val) swap(x, y);
4     if (rand() & 1) // 随机选择是否交换左右子节点
5         swap(t[x].ls, t[x].rs);
6     t[x].ls = merge(t[x].ls, y);
7     return x;
8 }

```

可以看到该实现方法唯一不同之处便是采用了随机数来实现合并，这样一来便可以省去 的相关计算。且平均时间复杂度亦为 $O(n \log n)$ ，详细证明可参考 [Randomized Heap](#)。

斜堆

斜堆是左偏树的自适应形式。当合并两个堆时，它无条件交换合并路径上的所有节点，以此试图维护平衡。根据均摊分析，自顶向下斜堆（top-down skew heap）插入，合并，删除最小值的复杂度为 $O(\log n)$ 。

例题

模板题

[luogu P3377【模板】左偏树（可并堆）](#)

[Monkey King](#)

[罗马游戏](#)

需要注意的是：

1. 合并前要检查是否已经在同一堆中。
2. 左偏树的深度可能达到 $O(n)$ ，因此找一个点所在的堆顶要用并查集维护，不能直接暴力跳父亲。（虽然很多题数据水，暴力跳父亲可以过……）（用并查集维护根时要保证原根指向新根，新根指向自己。）

► 罗马游戏参考代码

树上问题

[「APIO2012」派遣](#)

[「JLOI2015」城池攻占](#)

这类题目往往是每个节点维护一个堆，与儿子合并，依题意弹出、修改、计算答案，有点像线段树合并的类似题目。

► 城池攻占参考代码

[「SCOI2011」棘手的操作](#)

首先，找一个节点所在堆的堆顶要用并查集，而不能暴力向上跳。

再考虑单点查询，若用普通的方法打标记，就得查询点到根路径上的标记之和，最坏情况下可以达到 $O(n)$ 的复杂度。如果只有堆顶有标记，就可以快速地查询了，但如何做到呢？

可以用类似启发式合并的方式，每次合并的时候把较小的那个堆标记暴力下传到每个节点，然后把较大的堆的标记作为合并后的堆的标记。由于合并后有另一个堆的标记，所以较小的堆下传标记时要下传其标记减去另一个堆的标记。由于每个节点每被合并一次所在堆的大小至少乘二，所以每个节点最多被下放 $\log n$ 次标记，暴力下放标记的总复杂度就是 $O(n \log n)$ 。

再考虑单点加，先删除，再更新，最后插入即可。

然后是全局最大值，可以用一个平衡树/支持删除任意节点的堆（如左偏树）/multiset 来维护每个堆的堆顶。

所以，每个操作分别如下：

1. 暴力下传点数较小的堆的标记，合并两个堆，更新 size、tag，在 multiset 中删去合并后不在堆顶的那个原堆顶。
2. 删除节点，更新值，插入回来，更新 multiset。需要分删除节点是否为根来讨论一下。
3. 堆顶打标记，更新 multiset。
4. 打全局标记。
5. 查询值 + 堆顶标记 + 全局标记。
6. 查询根的值 + 堆顶标记 + 全局标记。
7. 查询 multiset 最大值 + 全局标记。

► 棘手的操作参考代码

[「BOI2004」Sequence 数字序列](#)

这是一道论文题，详见 [《黄源河 -- 左偏树的特点及其应用》](#)。

参考资料

-
1. [Self-Adjusting Heaps](#) ↩
-

本页面最近更新：2024/5/25 15:05:04，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[JiZiQian](#), [aofall](#), [CoelacanthusHex](#), [Early0v0](#), [Enter-tainer](#), [FFjet](#), [Henry-ZHR](#), [HeRaNO](#), [iamtwz](#), [Ir1d](#), [kenlig](#), [ksyx](#), [l1e1xx](#), [Marcythm](#), [ouuan](#), [Persdre](#), [shuzhouliu](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#), [YanJun-Zhao](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用

